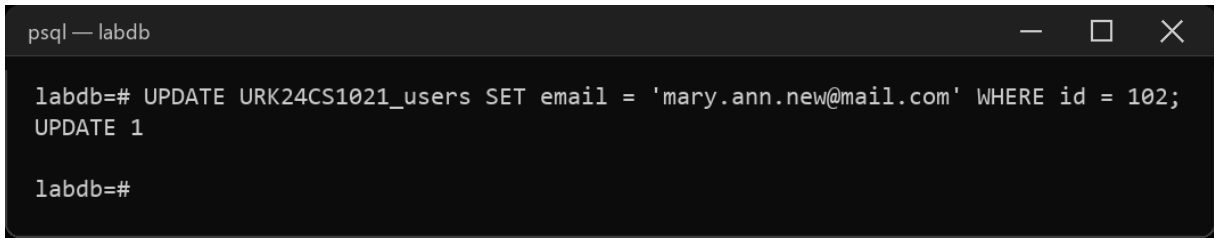


Ex. No. 1B Date: 24/07/2026	MANAGING TABLES USING DML, DCL AND TCL COMMANDS	URK24CS1021
Aim To execute all commands of Data Manipulation Language, Data Control Language, and Transaction Control Language to get the desired output.		
Description DML statements (INSERT/UPDATE/DELETE) manipulate row data. DCL statements (GRANT/REVOKE) control access privileges on database objects for specific roles. TCL statements (COMMIT/ROLLBACK/SAVEPOINT) control transaction boundaries — COMMIT makes changes permanent, SAVEPOINT marks a point to roll back to within an open transaction, and ROLLBACK TO undoes work back to that point without discarding the whole transaction.		
Questions		
1. Insert a new user into the "User" table.		
<pre>INSERT INTO URK24CS1021_users (id, name, email, password, phone) VALUES (103, 'Nina Patel', 'nina@mail.com', 'pass3', '9000000012');</pre>		
 The screenshot shows a terminal window titled 'psql — labdb'. The user enters the command 'INSERT INTO URK24CS1021_users (id, name, email, password, phone) VALUES (103, 'Nina Patel', 'nina@mail.com', 'pass3', '9000000012');'. The terminal response is 'INSERT 0 1', indicating that one row was inserted successfully.		
2. Update the email address of a user with UserID 102.		
<pre>UPDATE URK24CS1021_users SET email = 'mary.ann.new@mail.com' WHERE id = 102;</pre>		
 The screenshot shows a terminal window titled 'psql — labdb'. The user enters the command 'UPDATE URK24CS1021_users SET email = 'mary.ann.new@mail.com' WHERE id = 102;'. The terminal response is 'UPDATE 1', indicating that one row was updated successfully.		
3. Delete a user with the email "example@example.com".		
<pre>INSERT INTO URK24CS1021_users (id, name, email, password, phone) VALUES (104, 'Temp User', 'example@example.com', 'pass4', '9000000013'); DELETE FROM URK24CS1021_users WHERE email = 'example@example.com';</pre>		

psql — labdb

```
labdb=# INSERT INTO URK24CS1021_users (id, name, email, password, phone)
labdb(# VALUES (104, 'Temp User', 'example@example.com', 'pass4', '9000000013'));
INSERT 0 1

labdb=# DELETE FROM URK24CS1021_users WHERE email = 'example@example.com';
DELETE 1

labdb=#
```

4. Insert a new event into the "Event" table.

```
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description)
VALUES (402, 'Food Festival', '2023-11-05', '2023-11-05 12:00:00', 1, 'Street food
festival');
```

psql — labdb

```
labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description)
labdb(# VALUES (402, 'Food Festival', '2023-11-05', '2023-11-05 12:00:00', 1, 'Street food festival');
INSERT 0 1

labdb=#
```

5. Update the description of an event with EventID 401.

```
UPDATE URK24CS1021_event SET description = 'Annual comic convention, now with a
cosplay contest'
WHERE eventid = 401;
```

psql — labdb

```
labdb=# UPDATE URK24CS1021_event SET description = 'Annual comic convention, now with a cosplay contest'
labdb(# WHERE eventid = 401;
UPDATE 1

labdb=#
```

6. Grant SELECT privileges on the "User" table to a user named "john".

```
CREATE ROLE john LOGIN;
GRANT SELECT ON URK24CS1021_users TO john;
```

```
psql — labdb

labdb=# CREATE ROLE john LOGIN;
CREATE ROLE

labdb=# GRANT SELECT ON URK24CS1021_users TO john;
GRANT

labdb=#
```

7. Revoke INSERT privileges on the "Event" table from a user named "mary".

```
CREATE ROLE mary LOGIN;
GRANT INSERT ON URK24CS1021_event TO mary;
REVOKE INSERT ON URK24CS1021_event FROM mary;
```

```
psql — labdb

labdb=# CREATE ROLE mary LOGIN;
CREATE ROLE

labdb=# GRANT INSERT ON URK24CS1021_event TO mary;
GRANT

labdb=# REVOKE INSERT ON URK24CS1021_event FROM mary;
REVOKE

labdb=#
```

8. Create a new user with the username "jane" and grant them all privileges on the "Ticket" table.

```
CREATE ROLE jane LOGIN;
GRANT ALL PRIVILEGES ON URK24CS1021_ticket TO jane;
```

psql — labdb

```
labdb=# CREATE ROLE jane LOGIN;  
CREATE ROLE
```

```
labdb=# GRANT ALL PRIVILEGES ON URK24CS1021_ticket TO jane;  
GRANT
```

```
labdb=#
```

9. Grant EXECUTE privileges on a stored procedure to a role named "admin".

```
CREATE ROLE admin;  
CREATE OR REPLACE PROCEDURE URK24CS1021_ticket_count() LANGUAGE sql AS $$ SELECT 1;  
$$;  
GRANT EXECUTE ON PROCEDURE URK24CS1021_ticket_count() TO admin;
```

psql — labdb

```
labdb=# CREATE ROLE admin;  
CREATE ROLE
```

```
labdb=# CREATE OR REPLACE PROCEDURE URK24CS1021_ticket_count() LANGUAGE sql AS $$ SELECT 1; $$;  
CREATE PROCEDURE
```

```
labdb=# GRANT EXECUTE ON PROCEDURE URK24CS1021_ticket_count() TO admin;  
GRANT
```

```
labdb=#
```

10. Revoke all privileges from a user named "guest" on all tables in the schema.

```
CREATE ROLE guest LOGIN;  
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM guest;
```

psql — labdb

```
labdb=# CREATE ROLE guest LOGIN;  
CREATE ROLE
```

```
labdb=# REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM guest;  
REVOKE
```

```
labdb=#
```

11. Perform commit a transaction in the database.

```
BEGIN;  
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)  
VALUES (902, 401, 102, 80.00, 'confirmed', 'BCX002');  
COMMIT;
```

psql — labdb

```
labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (902, 401, 102, 80.00, 'confirmed', 'BCX002'));
labdb(# COMMIT;
BEGIN
INSERT 0 1
COMMIT

labdb=#
```

12. Perform roll back a transaction to a specific savepoint.

```
BEGIN;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
VALUES (903, 401, 103, 95.00, 'confirmed', 'BCX003');
SAVEPOINT sp1;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
VALUES (904, 401, 104, 95.00, 'confirmed', 'BCX004');
ROLLBACK TO SAVEPOINT sp1;
COMMIT;
SELECT ticketid FROM URK24CS1021_ticket WHERE ticketid IN (903, 904);
```

psql — labdb

```
labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (903, 401, 103, 95.00, 'confirmed', 'BCX003');
labdb(# SAVEPOINT sp1;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (904, 401, 104, 95.00, 'confirmed', 'BCX004');
labdb(# ROLLBACK TO SAVEPOINT sp1;
labdb(# COMMIT;
labdb(# SELECT ticketid FROM URK24CS1021_ticket WHERE ticketid IN (903, 904);
BEGIN
INSERT 0 1
SAVEPOINT
INSERT 0 1
ROLLBACK
COMMIT
 ticketid
-----
      903
(1 row)

labdb=#
```

13. Perform set a savepoint within a transaction.

```
BEGIN;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
VALUES (905, 401, 105, 70.00, 'confirmed', 'BCX005');
SAVEPOINT sp2;
SELECT COUNT(*) AS tickets_so_far FROM URK24CS1021_ticket;
COMMIT;
```

```
psql — labdb

labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (905, 401, 105, 70.00, 'confirmed', 'BCX005'));
labdb(# SAVEPOINT sp2;
labdb(# SELECT COUNT(*) AS tickets_so_far FROM URK24CS1021_ticket;
labdb(# COMMIT;
BEGIN
INSERT 0 1
SAVEPOINT
 tickets_so_far
-----
                4
(1 row)

COMMIT

labdb=#
```

14. Enable autocommit mode in the database.

```
\set AUTOCOMMIT on
```

```
psql — labdb

labdb=# \set AUTOCOMMIT on

labdb=#
```

15. Disable autocommit mode in the database.

```
\set AUTOCOMMIT off
```

A terminal window titled 'psql — labdb' with standard window controls (minimize, maximize, close). The terminal shows a PostgreSQL prompt 'labdb=#' followed by the command '\set AUTOCOMMIT off'. The output is another 'labdb=#' prompt.

```
psql — labdb

labdb=# \set AUTOCOMMIT off

labdb=#
```

Result

The queries were executed and the desired output was obtained.